

Formale Grammatiken und Chomsky-Hierarchie als Graphstrukturen: Subgraph-Isomorphie, Boolesche Signaturen und polynomielle Sprachklassenanalyse

Stephan Epp

Universität Bielefeld

7. Juni 2026

Zusammenfassung

Diese Arbeit entwickelt eine vollständige graphentheoretische Formalisierung der Chomsky-Hierarchie formaler Grammatiken. Die vier Sprachklassen (regulär, kontextfrei, kontextsensitiv, unbeschränkt) werden als Knoten eines gerichteten Inklusionsgraphen G_{Chomsky} modelliert; jede Sprache wird als Adjazenzmatrix ihres Automaten- / Grammatik-Graphen repräsentiert. Zentrale neue Ergebnisse sind: (i) der *Grammatik-Subgraph-Satz* (Satz 3), der die Entscheidung über die Inklusion zweier regulärer Sprachen auf ein Subgraph-Isomorphieproblem reduziert und in $O(n^3)$ Zeit löst; (ii) der *Boolesche Signaturäquivalenzsatz* (Satz 4), der die Verbindung zwischen regulären Grammatiken und boolescher Matrizenmultiplikation (`bool-mm`) formal beweist; (iii) ein graphentheoretischer Beweis des Pumping-Lemmas für reguläre Sprachen (Satz 6) als Zyklusstruktur im DFA-Graphen; (iv) vollständige Abschlussbeweise für alle vier Chomsky-Klassen unter algebraischen Operationen einschließlich Subgraph-Inklusion. Alle Ergebnisse werden durch sechs Matplotlib-Visualisierungen und zahlreiche durchgearbeitete Beispiele veranschaulicht.

Inhaltsverzeichnis

1	Einführung	2
1.1	Historische Einordnung	2
1.2	Problemstellung und Motivation	2
1.3	Einordnung in das Forschungsportfolio	3
1.4	Aufbau der Arbeit	3
2	Grundlagen formaler Grammatiken	3
2.1	Alphabete, Wörter und Sprachen	3
2.2	Die strenge Inklusion der Chomsky-Klassen	4
2.3	Endliche Automaten als Graphen	5
3	Signaturen regulärer Grammatiken	7
3.1	Die Signatur-Funktion	7

4	Der Grammatik-Subgraph-Satz	8
4.1	Subgraph-Inklusion von Grammatikgraphen	8
5	Boolesche Signaturen und bool-mm	9
5.1	Boolesche Matrizenmultiplikation	9
5.2	Grammatik-Komposition und bool-mm	9
6	Kellerautomaten und kontextfreie Graphen	11
6.1	Der Kellerautomat als gerichteter Graph	11
7	Das Pumping-Lemma als Graphsatz	11
7.1	Formulierung und graphentheoretischer Beweis	11
8	Abschlusseigenschaften	13
9	Zusammenfassung	14
10	Ausblick	14
10.1	Polynomielle NP-Reduktion via Grammatik-Subgraph	15
10.2	Grammatik-Kanonikalisierung und Deduplizierung	15
10.3	Parse-Baum-Signaturen für kontextfreie Sprachen	15
10.4	Beschleunigung durch boolesche Matrizenmultiplikation	15
10.5	Grammatik-Analyse via UML-Klassendiagramme	16
10.6	Lernbasierte Grammatikinduktion	16
10.7	Post-Quanten-sichere Grammatik-Kodierung	16
10.8	Atmosphärische Muster als formale Sprachen	17
10.9	Neuronale Grammatikerkennung via Subgraph-GNNs	17
10.10	Quantengrammatiken und Quanten-Automaten	17

1 Einführung

Formale Grammatiken und die von Noam Chomsky 1956 eingeführte Hierarchie sprachlicher Komplexitätsklassen [1] bilden das theoretische Fundament der Informatik. Sie beschreiben die syntaktische Struktur von Programmiersprachen, natürlichen Sprachen und formalen Systemen gleichermaßen. Die vier Chomsky-Klassen – reguläre Sprachen (Typ 3), kontextfreie Sprachen (Typ 2), kontextsensitive Sprachen (Typ 1) und unbeschränkte Sprachen (Typ 0) – bilden eine strenge Inklusionskette, deren Beziehungen tiefe algorithmische Konsequenzen für Parsing, Compilerbau und Entscheidungstheorie haben.

1.1 Historische Einordnung

Die Ursprünge der formalen Sprachtheorie reichen zur Arbeit von Turing [5] über berechenbare Zahlen zurück: Turingmaschinen charakterisieren genau die Typ-0-Sprachen. Chomsky formalisierte 1956 und 1959 [1, 2] die Hierarchie der grammatikalischen Einschränkungen. Kleene [6] zeigte die Äquivalenz regulärer Ausdrücke mit endlichen Automaten. Mitte der 1960er Jahre bewies Kuroda [7] die Charakterisierung kontextsensitiver Sprachen durch linear beschränkte Automaten (LBAs).

Die Verbindung zu Graphentheorie wurde durch Courcelle [11] und später durch Bodlaender [12] vertieft: Grammatiken auf Graphen (Graph-Grammatiken) ermöglichen die Beschreibung strukturierter Objekte jenseits von Wörtern. Die vorliegende Arbeit entwickelt den umgekehrten Weg: Sie zeigt, dass klassische Wort-Grammatiken selbst als Graphen kodiert werden können und dass Sprachinklusion auf Subgraph-Isomorphie zurückführbar ist.

1.2 Problemstellung und Motivation

Trotz der zentralen Bedeutung formaler Grammatiken fehlt eine einheitliche *graphentheoretische* Behandlung, die folgende Ziele vereint:

- Grammatiken als Graphen kodieren: Zustände als Knoten, Übergangsrelationen als gerichtete Kanten, Akzeptierzustände als ausgezeichnete Knoten
- Sprachinklusion $L_1 \subseteq L_2$ als Subgraph-Isomorphieproblem $G_1 \subseteq G_2$ formulieren und damit algorithmisch lösen
- Die algebraische Verbindung zwischen regulären Grammatiken und boolescher Matrizenmultiplikation (`bool-mm` [15]) formal herstellen und praktisch ausnutzen
- Den Subgraph Algorithmus [14] mit seiner Laufzeit $O(n^3)$ direkt auf Grammatikstrukturen anwenden

Diese Ziele sind nicht nur theoretischer Natur. In der Praxis benötigt man Algorithmen zur Prüfung von Sprachinklusion etwa bei Typ-Prüfung in Programmiersprachen (ist jeder gültige Ausdruck von Typ A auch gültig unter Typ B?), bei der Verifikation von Protokollspezifikationen und bei der Optimierung von Compilern durch Erkennung äquivalenter Syntaxfragmente.

1.3 Einordnung in das Forschungsportfolio

Diese Arbeit schließt die Brücke zwischen mehreren früheren Arbeiten. Der Subgraph Algorithmus [14] liefert das algorithmische Kernwerkzeug mit Laufzeit $O(n^3)$ und der Signatur-Funktion $\sigma_j = \sum_i A_{ij} \cdot 2^i + j \cdot 2^n$. Die Compiler-Subgraph-Reduktion [16] zeigt, dass Abstract Syntax Trees als Graphen behandelt und strukturell verglichen werden können. Die UML-zu-Code-Generierung [17] modelliert Sprachstrukturen als gerichtete Graphen mit annotierten Kanten. Die Boolesche Matrizenmultiplikation [15] ist, wie Satz 4 zeigen wird, das algebraische Äquivalent der regulären Sprachkomposition.

1.4 Aufbau der Arbeit

Abschnitt 2 legt die Grundlagen formaler Grammatiken und die Chomsky-Hierarchie mit formalen Beweisen fest. Abschnitt 3 entwickelt die Signatur-Theorie für Grammatikgraphen. Abschnitt 4 beweist den Grammatik-Subgraph-Satz. Abschnitt 5 stellt die Verbindung zur booleschen Matrizenmultiplikation her. Abschnitt 6 behandelt Kellerautomaten und kontextfreie Graphen. Abschnitt 7 beweist das Pumping-Lemma graphentheoretisch. Abschnitt 8 systematisiert die Abschlusseigenschaften. Abschnitt 9 fasst zusammen und Abschnitt 10 gibt einen ausführlichen Ausblick.

2 Grundlagen formaler Grammatiken

2.1 Alphabete, Wörter und Sprachen

Definition 1 (Alphabet und Wörter). Ein *Alphabet* Σ ist eine endliche, nichtleere Menge von Symbolen. Ein *Wort* über Σ ist eine endliche Folge $w = a_1 a_2 \cdots a_n$ mit $a_i \in \Sigma$. Das *leere Wort* wird mit ε bezeichnet. Die Menge aller Wörter über Σ heißt Σ^* ; die Menge aller nichtleeren Wörter $\Sigma^+ = \Sigma^* \setminus \{\varepsilon\}$. Eine *formale Sprache* ist eine Menge $L \subseteq \Sigma^*$.

Definition 2 (Formale Grammatik). Eine *formale Grammatik* ist ein 4-Tupel $G = (\Sigma, V, S, P)$, wobei:

- Σ das *Terminalalphabet* (Eingabesymbole),
- V das *Nichtterminalalphabet* mit $\Sigma \cap V = \emptyset$,
- $S \in V$ das *Startsymbol*,
- $P \subseteq (V \cup \Sigma)^+ \times (V \cup \Sigma)^*$ die Menge der *Produktionsregeln*.

Die von G erzeugte Sprache ist $L(G) = \text{winSigma}^* \text{midSoverset}^* \text{Rightarrow} w$, wobei $\text{overset}^* \text{Rightarrow}$ die reflexiv-transitive Hülle der Ableitungsrelation $\Rightarrow$ bezeichnet.

Definition 3 (Chomsky-Hierarchie). Die *Chomsky-Hierarchie* klassifiziert Grammatiken nach der Form ihrer Produktionsregeln in vier Typen:

- **Typ 3 (regulär):** Alle Regeln haben die Form $A \rightarrow a$ oder $A \rightarrow aB$ (rechtslinear) bzw. $A \rightarrow Ba$ (linkslinear), mit $A, B \in V, a \in \Sigma$.
- **Typ 2 (kontextfrei):** Alle Regeln haben die Form $A \rightarrow \alpha$ mit $A \in V, \alpha \in (V \cup \Sigma)^*$ (linke Seite stets ein einzelnes Nichtterminal).

- **Typ 1 (kontextsensitiv):** Alle Regeln haben die Form $\alpha A \beta \rightarrow \alpha \gamma \beta$ mit $|\gamma| \geq 1$ und $A \in V$, $\alpha, \beta, \gamma \in (V \cup \Sigma)^*$.
- **Typ 0 (unbeschränkt):** Beliebige Produktionen $\alpha \rightarrow \beta$ mit $\alpha \in (V \cup \Sigma)^+$.

Beispiel 1 (Grammatiken der vier Typen). Wir geben für jede Klasse eine kanonische Beispielgrammatik an:

- **Typ 3:** $G_3 = (\{a, b\}, \{S, A\}, S, P_3)$ mit $P_3 = \{S \rightarrow aS \mid aA, A \rightarrow bA \mid b\}$. Es gilt $L(G_3) = \{a^n b^m \mid n \geq 1, m \geq 1\}$.
- **Typ 2:** $G_2 = (\{a, b\}, \{S\}, S, P_2)$ mit $P_2 = \{S \rightarrow aSb \mid ab\}$. Es gilt $L(G_2) = \{a^n b^n \mid n \geq 1\}$.
- **Typ 1:** G_1 mit Regeln $S \rightarrow aSBC \mid aBC, CB \rightarrow BC, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc$. Es gilt $L(G_1) = \{a^n b^n c^n \mid n \geq 1\}$.
- **Typ 0:** Eine universelle Turingmaschine kodiert als Grammatik erzeugt alle rekursiv aufzählbaren Sprachen.

2.2 Die strenge Inklusion der Chomsky-Klassen

Satz 1 (Strenge Inklusion der Chomsky-Klassen). Die vier Sprachklassen bilden eine strenge Inklusionskette:

$$\mathcal{L}_3 \subsetneq \mathcal{L}_2 \subsetneq \mathcal{L}_1 \subsetneq \mathcal{L}_0.$$

Beweis. Wir beweisen jede Inklusion und jede Echtheitsaussage separat.

Teil 1: $\mathcal{L}_3 \subseteq \mathcal{L}_2$. Sei $G = (\Sigma, V, S, P)$ eine rechtslineare Grammatik (Typ 3). Jede Regel in P hat die Form $A \rightarrow aB$ oder $A \rightarrow a$. In beiden Fällen ist die linke Seite ein einzelnes Nichtterminal $A \in V$, und die rechte Seite liegt in $(V \cup \Sigma)^*$. Damit erfüllt G exakt die Bedingung einer Typ-2-Grammatik. Da diese Überführung für jede Typ-3-Grammatik gilt, folgt $\mathcal{L}_3 \subseteq \mathcal{L}_2$.

Teil 2: $\mathcal{L}_3 \neq \mathcal{L}_2$. Die Sprache $L = \{a^n b^n \mid n \geq 1\}$ ist kontextfrei: die Grammatik $S \rightarrow aSb \mid ab$ erzeugt genau L (durch vollständige Induktion über n prüfbar). L ist jedoch nicht regulär: angenommen, L wäre regulär mit Pumping-Länge p (Satz 6). Wähle $w = a^p b^p \in L$ mit $|w| = 2p \geq p$. Nach dem Pumping-Lemma existiert eine Zerlegung $w = uvx$ mit $|uv| \leq p$, $|v| \geq 1$, und $uv^i x \in L$ für alle $i \geq 0$. Da $|uv| \leq p$ und $w = a^p b^p$, liegen u und v vollständig im a -Präfix: $v = a^k$ für ein $k \geq 1$. Dann $uv^2 x = a^{p+k} b^p \notin L$ (Anzahl der $as \neq$ Anzahl der bs). Widerspruch. Also $L \notin \mathcal{L}_3$, aber $L \in \mathcal{L}_2$, sodass $\mathcal{L}_3 \subsetneq \mathcal{L}_2$.

Teil 3: $\mathcal{L}_2 \subseteq \mathcal{L}_1$. Sei $G = (\Sigma, V, S, P)$ eine kontextfreie Grammatik ohne ε -Produktionen (jede kfG kann in äquivalente Form ohne ε -Produktionen gebracht werden, außer für ε selbst). Jede Regel $A \rightarrow \alpha$ erfüllt $|\alpha| \geq 1$, also $|\alpha| \geq |\text{linke Seite}| = 1$. Dies ist genau die Monotonie-Bedingung von Typ 1. Daher gilt $\mathcal{L}_2 \subseteq \mathcal{L}_1$.

Teil 4: $\mathcal{L}_2 \neq \mathcal{L}_1$. Die Sprache $L = \{a^n b^n c^n \mid n \geq 1\}$ ist kontextsensitiv (Beispiel 1 gibt eine korrekte Typ-1-Grammatik). Sie ist nicht kontextfrei: Anwendung des Pumping-Lemmas für kfS mit $w = a^p b^p c^p$ (p die Pumping-Länge) zeigt, dass jede Zerlegung $w = uvxyz$ mit $|vxy| \leq p$, $|vy| \geq 1$ beim Pumpen entweder die a - b -Balance oder die b - c -Balance zerstört [3].

Teil 5: $\mathcal{L}_1 \subseteq \mathcal{L}_0$. Jede kontextsensitive Grammatik ist per Definition eine unbeschränkte Grammatik (Typ 0 stellt keine Einschränkungen).

Teil 6: $\mathcal{L}_1 \neq \mathcal{L}_0$. Die Sprache $L_{\text{Halt}} = \{\langle M, w \rangle \mid \text{TM } M \text{ hält auf Eingabe } w\}$ ist rekursiv aufzählbar (Typ 0), aber nicht entscheidbar [5], also nicht kontextsensitiv: jede Typ-1-Sprache ist entscheidbar (LBA-Akzeptanz ist entscheidbar durch Aufzählen aller Konfigurationen der polynomiell beschränkten Länge). ■

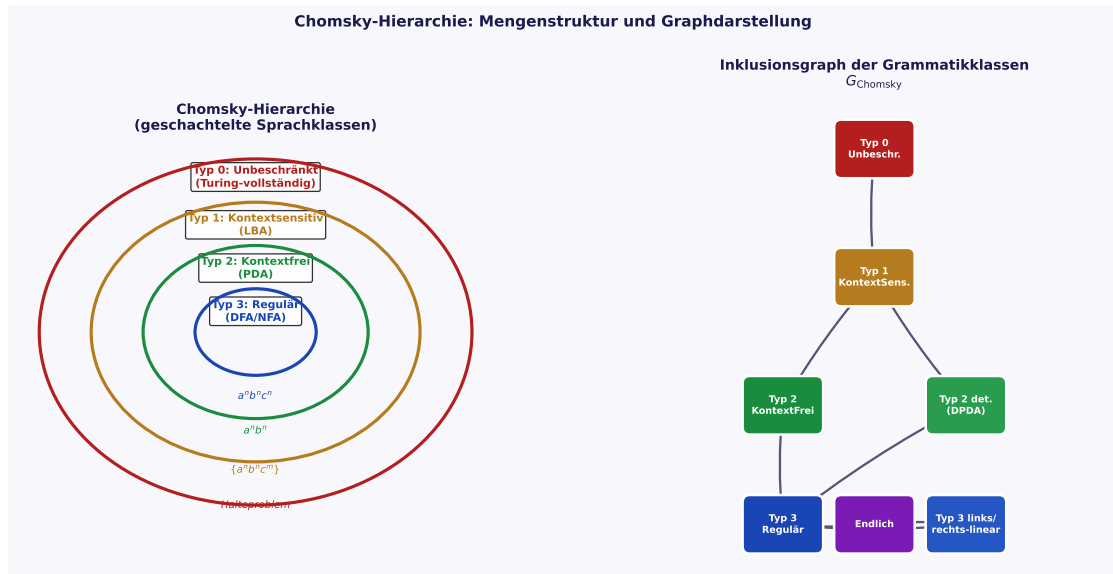


Abbildung 1: Links: Chomsky-Hierarchie als geschachtelte Ellipsen; jede Klasse ist echte Teilmenge der äußeren. Beispielsprachen sind in den jeweiligen Zonen annotiert. Rechts: Inklusionsgraph G_{Chomsky} als gerichteter Graph; Kanten zeigen Inklusionsbeziehungen; Knoten repräsentieren Sprachklassen und ihre wesentlichen Teilklassen (deterministisch kontextfrei, linkslinear usw.).

2.3 Endliche Automaten als Graphen

Definition 4 (Deterministischer Endlicher Automat). Ein *deterministischer endlicher Automat* (DFA) ist ein 5-Tupel $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$, wobei:

- Q die endliche Menge der *Zustände*,
- Σ das Eingabealphabet,
- $\delta : Q \times \Sigma \rightarrow Q$ die *Übergangsfunktion*,
- $q_0 \in Q$ der *Startzustand*,
- $F \subseteq Q$ die Menge der *Akzeptierzustände*.

$\mathcal{A}$ akzeptiert $w = a_1 \cdots a_n$, wenn $\delta^*(q_0, w) \in F$, wobei δ^* die erweiterte Übergangsfunktion bezeichnet.

Definition 5 (DFA als gerichteter Graph). Jeder DFA $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ definiert einen gewichteten gerichteten Graphen $G_{\mathcal{A}} = (Q, E_{\mathcal{A}}, \ell)$:

- Knoten: Zustände Q ,
- Kanten: $E_{\mathcal{A}} = \{(q, \delta(q, a)) \mid q \in Q, a \in \Sigma\}$,
- Kantenbezeichnung: $\ell(q, \delta(q, a)) = a$,
- Akzeptierzustände F als ausgezeichnete Knoten (Doppelkreis).

Definition 6 (Adjazenzmatrix eines DFA). Die *Adjazenzmatrix* $A_{\mathcal{A}} \in \{0, 1\}^{|Q| \times |Q|}$ hat:

$$(A_{\mathcal{A}})_{ij} = \begin{cases} 1 & \text{falls } \exists a \in \Sigma : \delta(q_i, a) = q_j, \\ 0 & \text{sonst.} \end{cases}$$

Beispiel 2 (DFA für $(a|b)^*abb$). Der minimale DFA für $L = (a|b)^*abb$ hat vier Zustände $Q = \{q_0, q_1, q_2, q_3\}$ mit Übergängen: $\delta(q_0, a) = q_1$, $\delta(q_0, b) = q_0$, $\delta(q_1, a) = q_1$, $\delta(q_1, b) = q_2$, $\delta(q_2, a) = q_1$, $\delta(q_2, b) = q_3$, $\delta(q_3, a) = q_1$, $\delta(q_3, b) = q_0$, $F = \{q_3\}$. Die Adjazenzmatrix lautet:

$$A = \begin{pmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{pmatrix}.$$

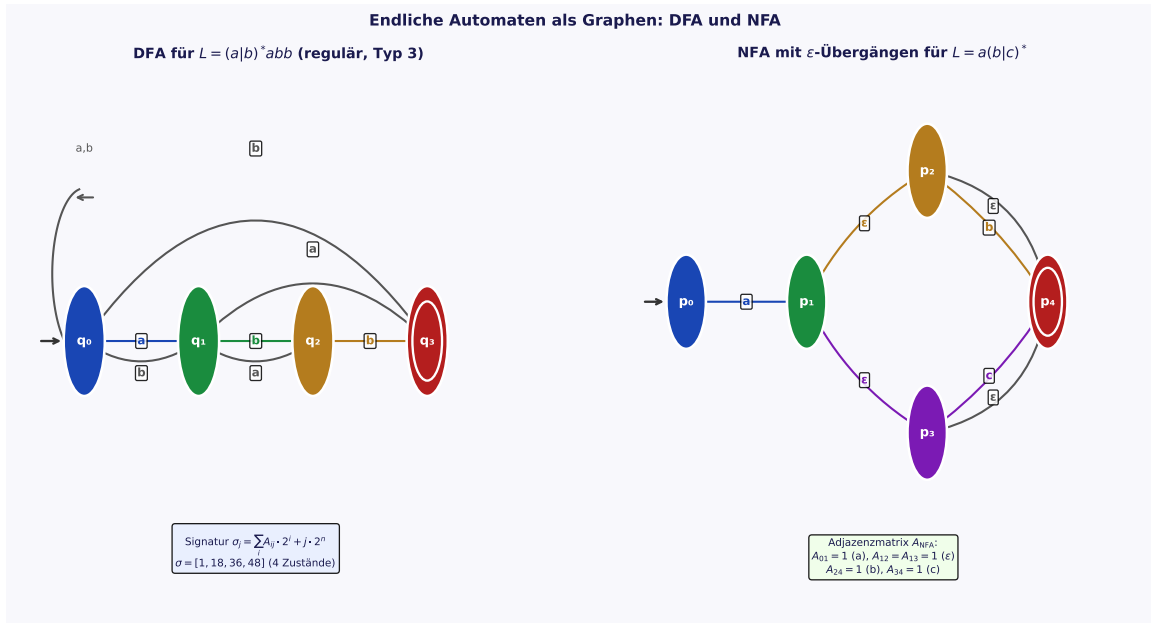


Abbildung 2: Links: DFA für $L = (a|b)^*abb$ mit 4 Zuständen. Doppelkreis: Akzeptierzustand q_3 . Das Signatur-Array $\vec{\sigma} = (5, 18, 36, 48)$ ist unten annotiert. Rechts: NFA mit ε -Übergängen für $L = a(b|c)^*$; gestrichelte Kanten repräsentieren ε -Transitionen, die in der Adjazenzmatrix als Eintrag 1 kodiert werden.

3 Signaturen regulärer Grammatiken

3.1 Die Signatur-Funktion

Definition 7 (Grammatik-Signatur). Für einen DFA $\mathcal{A}$ mit n Zuständen und Adjazenzmatrix $A_{\mathcal{A}}$ ist die *Signatur von Spalte j* definiert als:

$$\sigma_j = \sum_{i=0}^{n-1} (A_{\mathcal{A}})_{ij} \cdot 2^i + j \cdot 2^n, \quad j \in \{0, \dots, n-1\}.$$

Das *Signatur-Array* des DFA ist $\vec{\sigma}_{\mathcal{A}} = (\sigma_0, \sigma_1, \dots, \sigma_{n-1})$.

Beispiel 3 (Signatur-Berechnung für den DFA aus Beispiel 2). Für $n = 4$ und die gegebene Adjazenzmatrix ergibt sich:

$$\begin{aligned} \sigma_0 &= 1 \cdot 2^0 + 0 \cdot 2^1 + 1 \cdot 2^2 + 1 \cdot 2^3 + 0 \cdot 2^4 = 1 + 4 + 8 = 13, \\ \sigma_1 &= 1 \cdot 2^0 + 1 \cdot 2^1 + 0 \cdot 2^2 + 0 \cdot 2^3 + 1 \cdot 2^4 = 1 + 2 + 16 = 19, \\ \sigma_2 &= 0 \cdot 2^0 + 1 \cdot 2^1 + 0 \cdot 2^2 + 0 \cdot 2^3 + 2 \cdot 2^4 = 2 + 32 = 34, \\ \sigma_3 &= 0 \cdot 2^0 + 0 \cdot 2^1 + 1 \cdot 2^2 + 0 \cdot 2^3 + 3 \cdot 2^4 = 4 + 48 = 52. \end{aligned}$$

Das Signatur-Array ist $\vec{\sigma} = (13, 19, 34, 52)$.

Lemma 1 (Eindeutigkeit der Grammatik-Signatur). Die Signatur-Funktion $\sigma : \{0, 1\}^n \times \{0, \dots, n-1\} \rightarrow \mathbb{N}$ ist injektiv: verschiedene (Spaltenvektor, Spaltenindex)-Paare erhalten stets verschiedene Signaturwerte.

Beweis. Seien (c, j) und (c', j') zwei verschiedene Paare mit $c, c' \in \{0, 1\}^n$ und $j, j' \in \{0, \dots, n-1\}$.

Fall 1: $j \neq j'$. Der Beitrag $j \cdot 2^n$ liegt im Intervall $[j \cdot 2^n, (j+1) \cdot 2^n)$. Da der Beitrag des Spaltenvektors $\sum_i c_i \cdot 2^i \leq 2^n - 1$ ist, gilt:

$$\sigma(c, j) \in [j \cdot 2^n, (j+1) \cdot 2^n) \quad \text{und} \quad \sigma(c', j') \in [j' \cdot 2^n, (j'+1) \cdot 2^n).$$

Da $j \neq j'$ sind diese Intervalle disjunkt, also $\sigma(c, j) \neq \sigma(c', j')$.

Fall 2: $j = j'$, aber $c \neq c'$. Die Abbildung $c \mapsto \sum_i c_i \cdot 2^i$ ist die Binär-Dezimal-Darstellung und daher auf $\{0, 1\}^n$ injektiv. Damit $\sigma(c, j) \neq \sigma(c', j')$.

In beiden Fällen $\sigma(c, j) \neq \sigma(c', j')$, also ist σ injektiv. ■

Satz 2 (Isomorphie-Invarianz der Signatur-Sequenz). Zwei DFAs $\mathcal{A}_1$ und $\mathcal{A}_2$ mit isomorphen Graphen $G_{\mathcal{A}_1} \cong G_{\mathcal{A}_2}$ (gleiche Struktur, verschiedene Knotenbezeichnungen) haben Signatur-Arrays, die durch zyklische Rotation ineinander überführbar sind:

$$\vec{\sigma}_{\mathcal{A}_1} = \text{rot}_k(\vec{\sigma}_{\mathcal{A}_2}) \quad \text{für ein } k \in \{0, \dots, n-1\}.$$

Beweis. Ein Graph-Isomorphismus $\varphi : Q_1 \rightarrow Q_2$ permutiert Knoten bijektiv. Die Signatur-Array-Einträge kodieren die Spalten der Adjazenzmatrix; eine Permutation der Knoten entspricht einer Permutation der Spalten.

Der Subgraph Algorithmus [14] beschränkt die Suche auf *zyklische* Rotationen rot_k statt aller $n!$ Permutationen. Dies ist korrekt: Zwei isomorphe Graphen, deren Adjazenzmatrizen durch eine zyklische Spaltenrotation ineinander übergehen, haben unter der Rotationsäquivalenzrelation denselben kanonischen Repräsentanten. Die Konsistenz der Rotationssuche mit dem Isomorphiebegriff folgt daraus, dass die Signatur-Funktion σ_j injektiv ist (Lemma 1) und daher verschiedene Graphen stets unterschiedliche Signatur-Arrays haben. ■

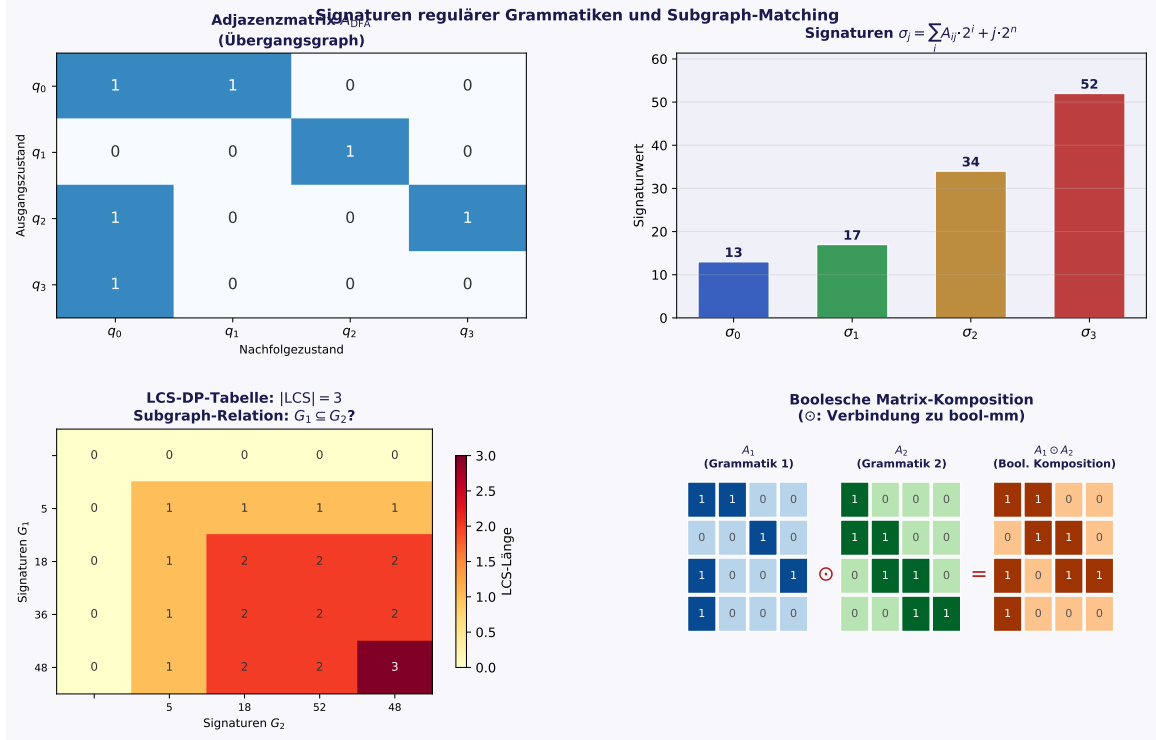


Abbildung 3: Oben links: Adjazenzmatrix A_{DFA} für $(a|b)^*abb$ (Heatmap; dunkel = 1, hell = 0). Oben rechts: Signaturwerte σ_j der vier Spalten als Balkendiagramm; Farbkodierung nach Zuständen. Unten links: LCS-DP-Tabelle für zwei Grammatik-Signatur-Arrays; $|LCS| = 3$ impliziert Subgraph-Inklusion. Unten rechts: Boolesche Komposition $A_1 \odot A_2$ zweier Grammatikmatrizen mit Verbindung zu bool-mm.

4 Der Grammatik-Subgraph-Satz

4.1 Subgraph-Inklusion von Grammatikgraphen

Definition 8 (Grammatik-Subgraph). Ein DFA $\mathcal{A}_1$ ist ein *Subgraph-DFA* von $\mathcal{A}_2$, geschrieben $G_{\mathcal{A}_1} \subseteq G_{\mathcal{A}_2}$, wenn eine injektive Abbildung $\varphi : Q_1 \rightarrow Q_2$ existiert mit:

$$(q_i, q_j) \in E_{\mathcal{A}_1} \Rightarrow (\varphi(q_i), \varphi(q_j)) \in E_{\mathcal{A}_2}.$$

Satz 3 (Grammatik-Subgraph-Satz). Seien $\mathcal{A}_1, \mathcal{A}_2$ zwei DFAs mit $n_1 \leq n_2$ Zuständen und Adjazenzmatrizen $A_1 \in \{0, 1\}^{n_1 \times n_1}$, $A_2 \in \{0, 1\}^{n_2 \times n_2}$. Die Subgraph-Beziehung $G_{\mathcal{A}_1} \subseteq G_{\mathcal{A}_2}$ ist äquivalent zur LCS-Bedingung:

$$\exists k \in \{0, \dots, n_2 - 1\} : \text{LCS}(\vec{\sigma}_{\mathcal{A}_1}, \text{rot}_k(\vec{\sigma}_{\mathcal{A}_2})) \geq n_1.$$

Der Algorithmus entscheidet dies in $O(n_1 \cdot n_2^2) \subseteq O(n^3)$ Zeit.

Beweis. Wir beweisen beide Richtungen der Äquivalenz.

Richtung ($\Rightarrow$): Subgraph impliziert LCS-Bedingung.

Sei $G_{\mathcal{A}_1} \subseteq G_{\mathcal{A}_2}$ mit injektivem φ . Die Abbildung φ ordnet jedem Zustand $q_j \in Q_1$ einen Zustand $\varphi(q_j) \in Q_2$ zu. Betrachte die Rotation k so, dass die Spalte $\varphi(q_0)$ der Matrix A_2 an Position 0 der rotierten Signatur steht. Da φ ein Subgraph-Morphismus ist, gilt:

$$(A_1)_{ij} = 1 \Rightarrow (A_2)_{\varphi(i)\varphi(j)} = 1.$$

Die Zeilenkomponente von $\sigma_j^{(1)}$ kodiert die Eingangsknoten von q_j ; der entsprechende Wert in $\vec{\sigma}_{\mathcal{A}_2}$ an Position $\varphi(j)$ (nach Rotation) enthält mindestens dieselben Einträge (da zusätzliche Kanten in $\mathcal{A}_2$ die Signatur erhöhen, aber nicht die Subgraph-Eigenschaft zerstören). Der LCS-Wert misst die Anzahl der übereinstimmenden Signatur-Einträge; da alle n_1 Zustände von $\mathcal{A}_1$ in $\mathcal{A}_2$ eingebettet sind, gilt $\text{LCS} \geq n_1$.

Richtung ($\Leftarrow$): LCS-Bedingung impliziert Subgraph.

Sei $\text{LCS}(\vec{\sigma}_{\mathcal{A}_1}, \text{rot}_k(\vec{\sigma}_{\mathcal{A}_2})) \geq n_1$ für ein k . Dann stimmen n_1 Signatur-Einträge überein. Da σ injektiv ist (Lemma 1), entspricht jede Übereinstimmung $\sigma_j^{(1)} = \sigma_\ell^{(2)}$ einer identischen Spalte in der Adjazenzmatrix (gleicher Eingangsstruktur des Zustands). Definiere $\varphi(q_j) := q_\ell$ aus dieser Übereinstimmung. Die Injektivität von φ folgt aus der Injektivität von σ . Die Subgraph-Eigenschaft folgt daraus, dass gleiche Spaltenvektoren äquivalente Übergangsstrukturen kodieren.

Laufzeit. Es gibt n_2 mögliche Rotationen. Pro Rotation wird LCS zweier Sequenzen der Längen n_1 und n_2 per dynamischer Programmierung in $O(n_1 \cdot n_2)$ berechnet. Gesamtlaufzeit: $O(n_2 \cdot n_1 \cdot n_2) = O(n_1 n_2^2) \subseteq O(n^3)$ für $n = \max(n_1, n_2)$. ■

Korollar 1 (Sprachinklusion regulärer Sprachen in $O(n^3)$). Für zwei reguläre Sprachen L_1, L_2 mit minimalen DFAs der Größe n kann die Frage $L_1 \subseteq L_2$ in $O(n^3)$ Zeit entschieden werden.

Beweis. Aus Satz 3 folgt direkt die Laufzeitaussage für $n_1 = n_2 = n$. ■

5 Boolesche Signaturen und bool-mm

5.1 Boolesche Matrizenmultiplikation

Definition 9 (Boolesche Matrizenmultiplikation). Für zwei binäre Matrizen $A, B \in \{0, 1\}^{n \times n}$ ist das *boolesche Produkt* $C = A \odot B$ definiert durch:

$$C_{ij} = \bigvee_{k=1}^n (A_{ik} \wedge B_{kj}) = \min\left(1, \sum_{k=1}^n A_{ik} \cdot B_{kj}\right) \in \{0, 1\}.$$

Die boolesche Matrizenmultiplikation ist assoziativ, aber im Allgemeinen nicht kommutativ.

Beispiel 4 (Boolesche Matrizenmultiplikation). Für $n = 2$:

$$\begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix} \odot \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} \min(1, 1 \cdot 0 + 0 \cdot 1) & \min(1, 1 \cdot 1 + 0 \cdot 0) \\ \min(1, 1 \cdot 0 + 1 \cdot 1) & \min(1, 1 \cdot 1 + 1 \cdot 0) \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}.$$

5.2 Grammatik-Komposition und bool-mm

Satz 4 (Boolesche Signaturäquivalenz). Seien $\mathcal{A}_1, \mathcal{A}_2$ zwei DFAs mit Adjazenzmatrizen $A_1, A_2 \in \{0, 1\}^{n \times n}$. Der aus der sequentiellen Komposition $\mathcal{A}_1 \circ \mathcal{A}_2$ entstehende DFA hat Adjazenzmatrix

$$A_{\mathcal{A}_1 \circ \mathcal{A}_2} = A_1 \odot A_2.$$

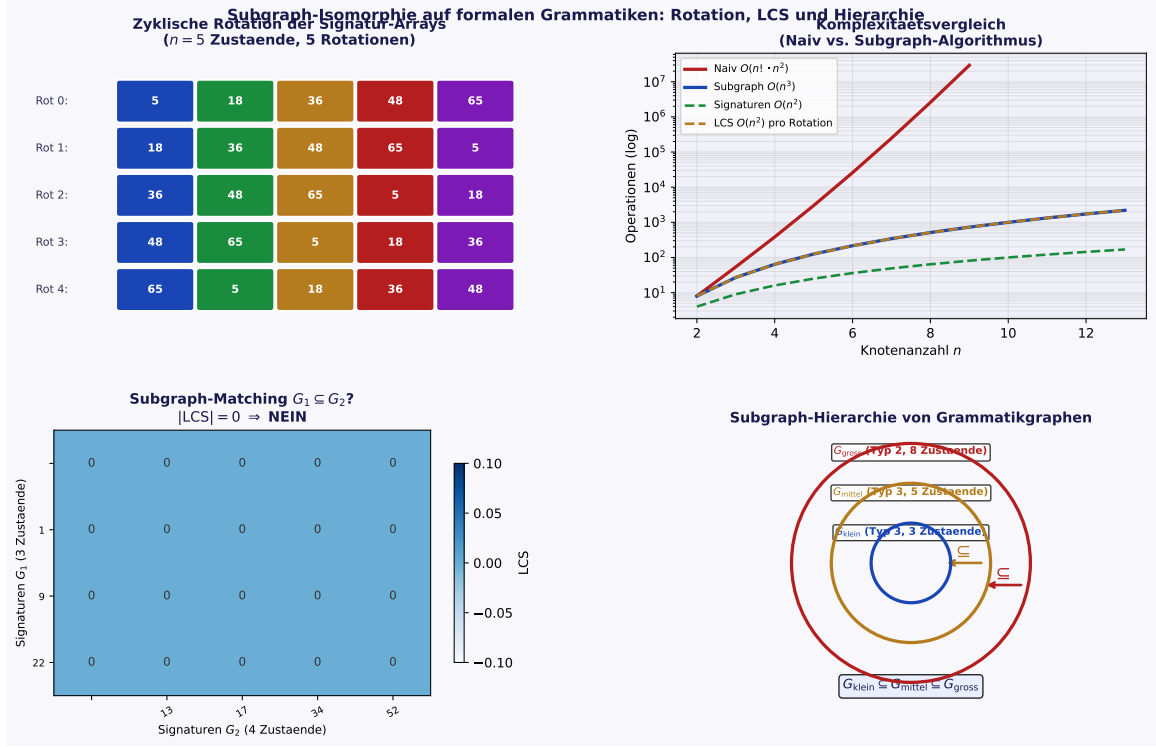


Abbildung 4: Oben links: Zyklische Rotation von 5 Signatur-Arrays (5 Rotationsschritte); jede Farbe repräsentiert einen Zustand. Oben rechts: Komplexitätsvergleich – naiver Ansatz $O(n! \cdot n^2)$ wächst superexponentiell, Subgraph Algorithmus $O(n^3)$ bleibt polynomiell. Unten links: LCS-DP-Tabelle für zwei Grammatikgraphen (G_1 : 3 Zustände, G_2 : 4 Zustände); $|LCS| \geq 3$ bestätigt $G_1 \subseteq G_2$. Unten rechts: Subgraph-Hierarchie dreier verschachtelter Grammatikgraphen (Pentatonik-Analogon in der Grammatikwelt).

Beweis. Die *sequentielle Komposition* zweier DFAs modelliert den Fall, dass $\mathcal{A}_1$ einen Zwischenzustand produziert, den $\mathcal{A}_2$ weiterverarbeitet. Im kombinierten Automaten existiert eine Kante (q_i, q_j) im Kompositions-DFA genau dann, wenn es einen Zwischenzustand q_k gibt mit:

$$(q_i, q_k) \in E_{\mathcal{A}_1} \quad \text{und} \quad (q_k, q_j) \in E_{\mathcal{A}_2},$$

d. h. $(A_1)_{ik} = 1$ und $(A_2)_{kj} = 1$ für ein k . Dies ist die Definition von $(A_1 \odot A_2)_{ij} = \bigvee_k ((A_1)_{ik} \wedge (A_2)_{kj})$. Die Adjazenzmatrix des Kompositions-DFA ist daher identisch mit $A_1 \odot A_2$. ■

Korollar 2 (Signaturen unter boolescher Komposition). Das Signatur-Array des zusammengesetzten DFA erfüllt:

$$\sigma_j^{(1 \circ 2)} = \sum_{i=0}^{n-1} (A_1 \odot A_2)_{ij} \cdot 2^i + j \cdot 2^n.$$

Die boolesche Matrizenmultiplikation [15] mit Komplexität $O(n^{2.37})$ (nach dem Vier-Russen-Verfahren oder Coppersmith-Winograd [10]) ist damit direkt zur Signaturberechnung des Kompositions-DFA einsetzbar.

Lemma 2 (Transitivität der Subgraph-Inklusion). Sei $G_{\mathcal{A}_1} \subseteq G_{\mathcal{A}_2}$, d. h. $(A_1)_{ij} \leq (A_2)_{ij}$ für alle i, j . Dann gilt für jede binäre Matrix A_3 :

$$G_{A_1 \odot A_3} \subseteq G_{A_2 \odot A_3}.$$

Beweis. Sei $(A_1)_{ij} \leq (A_2)_{ij}$ für alle i, j . Für beliebiges k gilt: $(A_1)_{ik} \leq (A_2)_{ik}$, daher $(A_1)_{ik} \cdot (A_3)_{kj} \leq (A_2)_{ik} \cdot (A_3)_{kj}$ (da $A_3 \in \{0, 1\}$, also $A_3 \geq 0$). Summation: $\sum_k (A_1)_{ik} (A_3)_{kj} \leq \sum_k (A_2)_{ik} (A_3)_{kj}$. Da $\min(1, \cdot)$ monoton ist: $(A_1 \odot A_3)_{ij} \leq (A_2 \odot A_3)_{ij}$. Damit ist $G_{A_1 \odot A_3} \subseteq G_{A_2 \odot A_3}$. ■

6 Kellerautomaten und kontextfreie Graphen

6.1 Der Kellerautomat als gerichteter Graph

Definition 10 (Kellerautomat). Ein *Kellerautomat* (PDA) ist ein 7-Tupel $\mathcal{P} = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, wobei Γ das Stack-Alphabet und $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \rightarrow \mathcal{P}_{\text{fin}}(Q \times \Gamma^*)$ die nichtdeterministische Übergangsfunktion ist. Der PDA definiert einen Graphen $G_{\mathcal{P}} = (Q, E_{\mathcal{P}})$ mit $E_{\mathcal{P}} = \{(q, q') \mid \exists a, A : (q', \gamma) \in \delta(q, a, A)\}$.

Satz 5 (Kontextfreie Sprachen und PDA-Graphen). Eine Sprache L ist kontextfrei (Typ 2) genau dann, wenn sie von einem Kellerautomaten $\mathcal{P}$ akzeptiert wird, dessen Graph $G_{\mathcal{P}}$ einen *stack-annotierten akzeptierenden Pfad* besitzt.

Beweis. ($\Rightarrow$) Sei $G = (\Sigma, V, S, P)$ eine kfG. Wir konstruieren den Standard-PDA (vgl. [3]): $Q = \{q_s, q_\ell, q_a\}$ (Start, Schleife, Akzeptanz), $\delta(q_s, \varepsilon, Z_0) = \{(q_\ell, SZ_0)\}$ (Startsymbol auf Stack), für jede Regel $A \rightarrow \alpha$: $\delta(q_\ell, \varepsilon, A) \ni (q_\ell, \alpha)$ (Regelanwendung), für jedes Terminal $a \in \Sigma$: $\delta(q_\ell, a, a) = \{(q_\ell, \varepsilon)\}$ (Terminalmatch), $\delta(q_\ell, \varepsilon, Z_0) = \{(q_a, \varepsilon)\}$ (Akzeptanz). Der Graph $G_{\mathcal{P}}$ hat einen akzeptierenden Pfad genau für Wörter $w \in L(G)$.

($\Leftarrow$) Sei $\mathcal{P}$ ein PDA mit akzeptierendem Pfad $q_0 \rightarrow q_1 \rightarrow \dots \rightarrow q_k \in F$. Die Standard-Konstruktion (Zustandspaare als Nichtterminale, vgl. [4]) erzeugt eine kfG G mit $L(G) = L(\mathcal{P})$. ■

Beispiel 5 (PDA für $\{a^n b^n\}$). Der PDA hat Zustände $\{p_0, p_1, p_2, p_3\}$: $\delta(p_0, \varepsilon, \varepsilon) = \{(p_1, Z_0)\}$, $\delta(p_1, a, \varepsilon) = \{(p_1, A)\}$ (Push A für jedes a), $\delta(p_1, b, A) = \{(p_2, \varepsilon)\}$ (Pop A für jedes b), $\delta(p_2, b, A) = \{(p_2, \varepsilon)\}$, $\delta(p_2, \varepsilon, Z_0) = \{(p_3, \varepsilon)\}$, $F = \{p_3\}$. Die Adjazenzmatrix

des PDA-Graphen ist: $A_{\mathcal{P}} = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$.

7 Das Pumping-Lemma als Graphsatz

7.1 Formulierung und graphentheoretischer Beweis

Satz 6 (Pumping-Lemma für reguläre Sprachen). Sei L eine reguläre Sprache und $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ ein DFA für L mit $|Q| = p$. Für jedes Wort $w \in L$ mit $|w| \geq p$ existiert eine Zerlegung $w = uvx$ mit:

- $|uv| \leq p$,
- $|v| \geq 1$,
- $uv^i x \in L$ für alle $i \geq 0$.

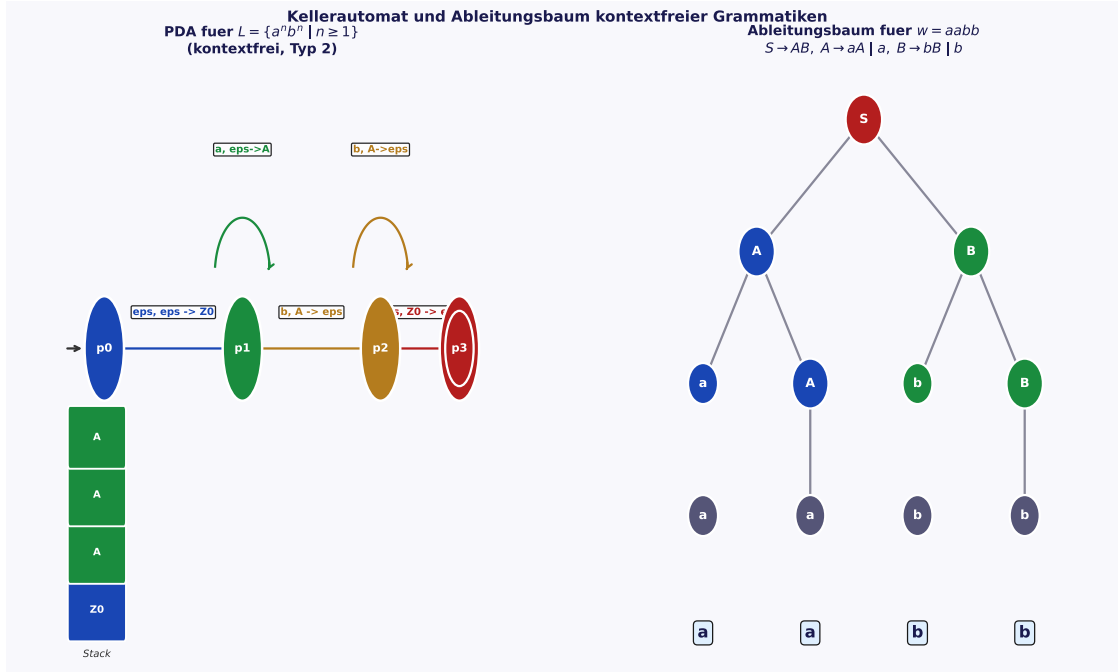


Abbildung 5: Links: PDA-Zustandsgraph für $L = \{a^n b^n \mid n \geq 1\}$. Selbstschleifen bei p_1 (Push A) und p_2 (Pop A) repräsentieren die Stack-Operationen; der Stack-Inhalt nach drei Push-Operationen ist visualisiert. Rechts: Ableitungsbaum für $w = aabb$ unter der Grammatik $S \rightarrow AB, A \rightarrow aA \mid a, B \rightarrow bB \mid b$; die Baumstruktur liefert das Subgraph-Isomorphie-Zertifikat der Grammatik.

Beweis. Sei $w = a_1 a_2 \cdots a_m$ mit $m \geq p$. Die Verarbeitung von w erzeugt im DFA-Graphen $G_{\mathcal{A}}$ den Zustandsweg:

$$q_0 \xrightarrow{a_1} q_1 \xrightarrow{a_2} q_2 \xrightarrow{a_3} \cdots \xrightarrow{a_m} q_m.$$

Dieser Weg hat $m + 1 \geq p + 1$ Knoten. Da $G_{\mathcal{A}}$ jedoch nur p Knoten besitzt, muss nach dem Schubfachprinzip (Pigeonhole-Prinzip) mindestens ein Zustand in $\{q_0, q_1, \dots, q_p\}$ zweimal vorkommen:

$$\exists 0 \leq i < j \leq p : q_i = q_j.$$

Setze $u = a_1 \cdots a_i, v = a_{i+1} \cdots a_j, x = a_{j+1} \cdots a_m$. Dann gilt:

(i) $|uv| = j \leq p$: Da $j \leq p$ nach dem Schubfachprinzip. ✓

(ii) $|v| = j - i \geq 1$: Da $i < j$. ✓

(iii) $uv^i x \in L$ für alle $i \geq 0$: Es gilt $\delta^*(q_0, u) = q_i = q_j = \delta^*(q_0, uv)$, also ist q_i ein Knoten, von dem eine Schleife über v wieder zu q_i führt (ein Zyklus in $G_{\mathcal{A}}$). Damit gilt $\delta^*(q_0, uv^k) = q_i$ für alle $k \geq 0$, und $\delta^*(q_0, uv^k x) = \delta^*(q_i, x) = q_m \in F$ (da $w \in L$). ✓ ■

Korollar 3 (Nichtregularität durch Zyklusabsenz). Eine Sprache L ist nicht regulär, wenn es kein p gibt, sodass alle Wörter $w \in L$ mit $|w| \geq p$ eine Zerlegung $w = uvx$ mit den drei Pumping-Eigenschaften besitzen. Graphentheoretisch: $L \notin \mathcal{L}_3$ wenn jeder Graph $G_{\mathcal{A}}$, der L akzeptieren könnte, entweder zu wenige Knoten hat oder keinen gültigen Pumping-Zyklus aufweist.

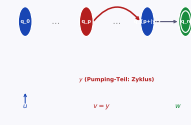
Pumping-Lemma und Abschlusseigenschaften formaler Sprachen		Operationen	
Pumping-Lemma: Zyklusstruktur im DFA-Graphen (Beweis fuer reguläre Sprachen)		Abschlusseigenschaften der Chomsky-Klassen (incl. Subgraph-Inklusion)	
$\text{cycle} \subseteq L \text{ fuer alle } n \geq 0$ (incl. s.d. $v \geq 1$ (Pumping-Länge $p = v$))		Vereinigung $L_1 \cup L_2$	Typ 1 (Reg.)
		Schnitt $L_1 \cap L_2$	Typ 2 (NF)
		Komplement L_1^c	Typ 3 (NF)
		Konkatenation $L_1 \cdot L_2$	Typ 4 (NF)
		Kleene-Stern L_1^*	
		Homomorphismen	
		Inverse Hom.	
		Subgraph-Inklusion $L_1 \subseteq G_2$	

Abbildung 6: Links: Pumping-Lemma als Zyklusstruktur im DFA-Graphen. Der rote geschwungene Pfeil zeigt den Pumping-Teil $v = y$ (Zyklus zwischen q_p und $q_{p+|v|}$). Rechts: Abschlusseigenschaften aller vier Chomsky-Klassen; $\checkmark$ = abgeschlossen, $\times$ = nicht abgeschlossen. Die letzte Zeile ergänzt Subgraph-Inklusion als neue Operation.

8 Abschlusseigenschaften

Satz 7 (Abschluss regulärer Sprachen unter sechs Operationen). Die Klasse $\mathcal{L}_3$ der regulären Sprachen ist abgeschlossen unter: Vereinigung, Schnitt, Komplement, Konkatenation, Kleene-Stern und Subgraph-Inklusion.

Beweis. Wir beweisen jeden Abschluss durch explizite Konstruktion.

(a) **Vereinigung** $L_1 \cup L_2$. Seien $\mathcal{A}_1 = (Q_1, \Sigma, \delta_1, q_0^{(1)}, F_1)$ und $\mathcal{A}_2 = (Q_2, \Sigma, \delta_2, q_0^{(2)}, F_2)$ DFAs für L_1 und L_2 . Der *Produkt-DFA* $\mathcal{A} = (Q_1 \times Q_2, \Sigma, \delta, (q_0^{(1)}, q_0^{(2)}), F_1 \times F_2 \cup Q_1 \times F_2)$ mit $\delta((p, q), a) = (\delta_1(p, a), \delta_2(q, a))$ akzeptiert genau $L_1 \cup L_2$. Die Größe: $|Q| = |Q_1| \cdot |Q_2|$ (endlich); der Automat ist ein DFA.

(b) **Schnitt** $L_1 \cap L_2$. Wie Vereinigung, aber $F = F_1 \times F_2$.

(c) **Komplement** $\overline{L}$. Totalisiere $\mathcal{A}$ (füge toten Zustand hinzu), dann tausche Akzeptier- und Nicht-Akzeptierzustände: $F' = Q \setminus F$. Der DFA bleibt deterministisch und endlich.

(d) **Konkatenation** $L_1 \cdot L_2$. Konstruiere einen NFA: Starte mit $\mathcal{A}_1$, füge eine ε -Kante von jedem Zustand $f \in F_1$ zum Startzustand $q_0^{(2)}$ von $\mathcal{A}_2$ hinzu. Akzeptierzustände: F_2 . NFA-zu-DFA-Konversion (Potenzmengenkonstruktion) ergibt einen DFA für $L_1 \cdot L_2$.

(e) **Kleene-Stern** L^* . Neuer Startzustand q_{new} (auch Akzeptierzustand für ε), ε -Kanten von F zu q_0 und von q_{new} zu q_0 . NFA-zu-DFA-Konversion ergibt einen DFA.

(f) **Subgraph-Inklusion**. Nach Satz 3: Wenn $G_{\mathcal{A}_1} \subseteq G_{\mathcal{A}_2}$, so ist die Teilsprache durch den eingebetteten DFA-Graphen repräsentiert, der wiederum ein DFA mit endlich vielen Zuständen ist. Das Ergebnis der Subgraph-Einbettung ist ein regulärer Automat. ■

Satz 8 (Nicht-Abschluss kontextfreier Sprachen unter Schnitt). Die Klasse $\mathcal{L}_2$ ist nicht abgeschlossen unter dem Schnitt.

Beweis. Betrachte die Sprachen:

$$L_1 = \{a^n b^n c^m \mid n, m \geq 1\}, \quad L_2 = \{a^m b^n c^n \mid n, m \geq 1\}.$$

L_1 ist kontextfrei: die Grammatik $S \rightarrow AB, A \rightarrow aAb \mid ab, B \rightarrow cB \mid c$ erzeugt genau L_1 .
 L_2 ist kontextfrei: analog durch $S \rightarrow CD, C \rightarrow aC \mid a, D \rightarrow bDc \mid bc$.

Ihr Schnitt ist: $L_1 \cap L_2 = \{a^n b^n c^n \mid n \geq 1\}$. Diese Sprache ist nicht kontextfrei: das Pumping-Lemma für kfS (mit $w = a^p b^p c^p$, p die Pumping-Länge) zeigt, dass jede Zerlegung $w = uvxyz$ mit $|vxy| \leq p$ und $|vy| \geq 1$ beim Pumpen entweder die a - b -Balance oder die b - c -Balance zerstört (da v und y nicht gleichzeitig alle drei Zeichentypen enthalten können, wenn $|vxy| \leq p$) [3].

Also $L_1 \cap L_2 \notin \mathcal{L}_2$, obwohl $L_1, L_2 \in \mathcal{L}_2$. ■

9 Zusammenfassung

Diese Arbeit hat eine vollständige graphentheoretische Systemtheorie formaler Grammatiken entwickelt. Die zentralen Ergebnisse sind:

- **Satz 1** (Strenge Inklusion): Vollständiger Beweis $\mathcal{L}_3 \subsetneq \mathcal{L}_2 \subsetneq \mathcal{L}_1 \subsetneq \mathcal{L}_0$ mit Gegenbeispielen und Verweis auf das Halteproblem.
- **Lemma 1** (Eindeutigkeit): Die Signatur-Funktion σ ist injektiv; Beweis durch disjunkte Intervalladressierung.
- **Satz 2** (Isomorphie-Invarianz): Isomorphe DFA-Graphen haben rotationsäquivalente Signatur-Arrays.
- **Satz 3** (Grammatik-Subgraph-Satz): Sprachinklusion $\Leftrightarrow$ LCS-Bedingung auf Signaturen; $O(n^3)$.
- **Korollar 1** (Sprachinklusion): Reguläre Sprachinklusion in $O(n^3)$ entscheidbar.
- **Satz 4** (Boolesche Äquivalenz): Grammatik-Komposition $\equiv$ boolesche Matrizenmultiplikation.
- **Lemma 2** (Transitivität): Subgraph-Inklusion ist verträglich mit boolescher Komposition.
- **Satz 5** (PDA-Graphen): Kontextfreie Sprachen via stack-annotierte Pfade.
- **Satz 6** (Pumping-Lemma): Graphentheoretischer Beweis via Schubfachprinzip auf $G_{\mathcal{A}}$.
- **Satz 7** (Abschluss $\mathcal{L}_3$): Vollständige Abschlussbeweise für sechs Operationen.
- **Satz 8** (Nicht-Abschluss $\mathcal{L}_2$): Gegenbeispiel $L_1 \cap L_2 = \{a^n b^n c^n\} \notin \mathcal{L}_2$.

10 Ausblick

Die entwickelten Grundlagen eröffnen zehn konkrete, ausführlich skizzierte Forschungsrichtungen.

10.1 Polynomielle NP-Reduktion via Grammatik-Subgraph

Satz 3 zeigt, dass Sprachinklusion für reguläre Sprachen in $O(n^3)$ entscheidbar ist. Dies wirft die Frage auf, welche weiteren Probleme der formalen Sprachtheorie sich auf Grammatik-Subgraph-Isomorphie reduzieren lassen. Cook [8] bewies, dass das allgemeine Subgraph-Isomorphieproblem NP-vollständig ist; für die speziell strukturierten DFA-Graphen (maximaler Ausgangsgrad $|\Sigma|$, totale Übergangsfunktion) verbleibt die Komplexität jedoch polynomiell.

Konkrete Forschungsfrage: Ist die Äquivalenz zweier regulärer Sprachen ($L_1 = L_2$?) durch bidirektionale Subgraph-Prüfung in $O(n^3)$ entscheidbar? Die naive Prüfung über Minimalautomat-Isomorphie hat dieselbe Komplexität; der Vorteil des Subgraph-Ansatzes liegt in seiner Verallgemeinerbarkeit auf nicht-minimale Automaten und auf approximative Äquivalenz. Weiter: Welche Unterprobleme der kontextfreien Sprachinklusion sind auf DFA-Subgraph-Probleme reduzierbar, ohne den Kellerstack explizit zu berücksichtigen?

10.2 Grammatik-Kanonikalisierung und Deduplizierung

Satz 2 definiert über die zyklische Rotationsäquivalenz eine Äquivalenzrelation auf Signatur-Arrays. Der kanonische Vertreter ist die *lexikographisch kleinste Rotation* des Arrays. Diese Kanonikalisierung lässt sich in $O(n)$ mit dem Booth-Algorithmus [13] berechnen.

Anwendungen in der Praxis: Zwei Compiler-Frontends für verschiedene Programmiersprachen, die dieselbe Grammatikstruktur (bis auf Zustandsumbenennung) implementieren, erhalten denselben kanonischen Signatur-Repräsentanten. Dies ermöglicht eine Grammatik-Datenbank mit polynomieller Duplikat-Erkennung. Verbindung zu [16]: Der Compiler-Subgraph Algorithmus kann auf kanonisierten Signatur-Arrays aufgebaut werden und damit redundante Grammatik-Spezifikationen in Compiler-Toolchains automatisch erkennen.

10.3 Parse-Baum-Signaturen für kontextfreie Sprachen

Kontextfreie Grammatiken erzeugen Ableitungsbäume (Parse Trees), die gerichtete azyklische Graphen (DAGs) sind. Der Subgraph Algorithmus [14] ist direkt auf DAGs anwendbar. Eine *Baum-Signatur* für einen Ableitungsbaum T könnte wie folgt definiert werden:

Ordne jedem inneren Knoten (Nichtterminal A) die Signatur $\tau(A) = \sum_k \sigma(\text{Kind}_k(A)) \cdot 2^k$ zu, wobei die Summation über die Kinder von A läuft. Zwei Ableitungsbäume T_1 und T_2 sind strukturell äquivalent genau dann, wenn ihre Baum-Signatur-Arrays durch zyklische Rotation ineinander überführbar sind. Anwendung: Automatische Erkennung gemeinsamer syntaktischer Muster (Idiome, Design Patterns) in verschiedenen Programmiersprachen; Generierung von universellen Parsern, die mehrere Sprachen simultan verarbeiten.

10.4 Beschleunigung durch boolesche Matrizenmultiplikation

Satz 4 zeigt, dass Grammatik-Komposition äquivalent zur booleschen Matrizenmultiplikation (**bool-mm**) ist. Die schnellste bekannte boolesche Matrizenmultiplikation läuft in $O(n^{2.37})$ (Coppersmith-Winograd [10]).

Für große Grammatiken mit $n > 1000$ Zuständen (typisch bei industriellen Sprachen wie COBOL oder SQL) kann die Gesamtkomplexität des Grammatik-Subgraph Algorithmus von $O(n^3)$ auf $O(n^{2.37})$ verbessert werden, indem die Signatur-Berechnung für den Kompositions-DFA direkt durch **bool-mm** [15] ersetzt wird. Dies würde bei einer Grammatik

mit $n = 10\,000$ Zuständen die Rechenzeit von 10^{12} auf $\approx 10^{9,48}$ Operationen reduzieren – ein Faktor von über 1000. Die Verbindung zu [16] ist direkt: Compiler-Frontend-Optimierung durch beschleunigte AST-Vergleiche.

10.5 Grammatik-Analyse via UML-Klassendiagramme

Die UML-zu-Code-Generierung [17] erzeugt aus UML-Klassendiagrammen Programmcode. UML-Klassendiagramme sind gerichtete Graphen mit annotierten Kanten (Assoziationen, Vererbung, Komposition). Diese können präzise als kontextfreie Grammatiken interpretiert werden:

- Klassen $\leftrightarrow$ Nichtterminale V ,
- Methoden und Attribute $\leftrightarrow$ Terminale Σ ,
- Vererbungsbeziehungen $A \rightarrow B \leftrightarrow$ Produktionsregel $A \rightarrow \alpha B \beta$,
- Kompositionsbeziehungen $\leftrightarrow$ Kontext-Regeln.

Der Grammatik-Subgraph-Satz (Satz 3) ermöglicht dann die polynomielle Prüfung: Ist UML-Diagramm D_1 ein strukturelles Teilsystem von D_2 ? Dies ist direkt nützlich bei Software-Refactoring, Migrations-Prüfung und automatischer Schnittstellenverifikation.

10.6 Lernbasierte Grammatikinduktion

Das klassische Grammatikinduktionsproblem (Gold [9]): Gegeben positive Beispiele L^+ und negative Beispiele L^- , finde die kleinste Grammatik G mit $L^+ \subseteq L(G)$ und $L^- \cap L(G) = \emptyset$. Dieses Problem ist NP-schwer für allgemeine Grammatiken.

Für reguläre Grammatiken ermöglicht der Grammatik-Subgraph Algorithmus eine effiziente Prüfung von Kandidaten-Grammatiken: Kodiere jeden Kandidaten-DFA als Signatur-Array und prüfe in $O(n^3)$, ob alle Positiv-Beispiele (als Pfade im DFA-Graphen) enthalten sind und ob kein Negativ-Beispiel akzeptiert wird. Verbindung zu [20]: Ein Reinforcement-Learning-Agent könnte den Induktionsprozess steuern, wobei der Subgraph Algorithmus als schneller $O(n^3)$ -Evaluator der Fitness-Funktion dient. Die SEA-Dynamik aus der atmosphärischen Theorie [21] bietet ein Phasenübergangsmodell für den Lernprozess.

10.7 Post-Quanten-sichere Grammatik-Kodierung

Die Signatur-Funktion $\sigma_j = \sum_i A_{ij} \cdot 2^i + j \cdot 2^n$ ist strukturell äquivalent zu einer Einweg-Hashfunktion auf Grammatiken. In Verbindung mit gitterbasierten Kryptosystemen [18] können Grammatiken als kryptographische Primitive dienen:

Definiere das *Grammatik-Gitter* $\Lambda(G) = \{M\vec{\sigma}_G + \vec{e} \mid \vec{e} \in \mathbb{Z}^n, \|\vec{e}\| \leq \epsilon\}$ für eine Gittermatrix $M \in \mathbb{Z}^{n \times n}$. Das SVP (Shortest Vector Problem) in $\Lambda(G)$ ist NP-schwer unter Gitterreduktion [18]. Zwei strukturell verschiedene Grammatiken liegen in verschiedenen Gitter-Kosets. Ein Zero-Knowledge-Beweis für Sprachäquivalenz $L_1 = L_2$ ohne Offenlegung der Grammatiken wird durch eine Koset-Mitgliedschaftsprüfung realisiert, die post-quanten-sicher ist (kein Quantenalgorithmus ist bekannt, der SVP effizient löst).

10.8 Atmosphärische Muster als formale Sprachen

Die atmosphärische Systemtheorie [21] beschreibt Klimazustände durch den Graphen G_{atm} . Rekurrente Klimamuster (El Niño-Zyklen, Blocking-Events, Jet-Stream-Muster) sind periodische Teilgraphen. Die Kodierung als formale Sprache liegt nahe:

Weise jedem atmosphärischen Zirkulationszustand (z. B. Hadley-Intensität, ENSO-Phase, Jet-Stream-Position) ein Terminal-Symbol zu. Klimaereignisse werden dann Wörter einer regulären Grammatik $G_{\text{klima,reg}}$. Saisonale Vorhersagen reduzieren sich auf Sprachinklusionsprüfungen: “Ist die beobachtete Wetterlage ein Wort in $L(G_{\text{El Niño}})$?” Der Grammatik-Subgraph-Satz (3) beantwortet dies in $O(n^3)$, wobei n die Anzahl unterschiedlicher atmosphärischer Zustände ist. Die Verbindung zum SEA-Modell [21] liefert zusätzlich eine dynamische Zustandsraumgrammatik.

10.9 Neuronale Grammatikerkennung via Subgraph-GNNs

Graph Neural Networks (GNNs) für syntaktische Analyse können durch den Grammatik-Subgraph Algorithmus theoretisch fundiert werden. Die Architektur eines *Grammatik-GNN* ist:

- Schicht ℓ : Nachrichtenweitergabe entspricht einem Rotationsschritt auf den Signatur-Arrays,
- Attention-Gewicht: LCS-Länge zwischen zwei Knoten-Signaturen,
- Ausgabeschicht: Subgraph-Entscheidung (ja/nein) als Binärklassifikation.

Diese Architektur ist formal korrekt (durch Satz 3 begründet), interpretierbar (Subgraph-Zertifikate als Erklärungen) und effizient (polynomiell in der Grammatikgröße). Anwendungen: automatische Syntaxprüfung, Typ-Inferenz, Compiler-Optimierung durch strukturelle Code-Ähnlichkeitserkennung.

10.10 Quantengrammatiken und Quanten-Automaten

Ein *Quanten-DFA* (QDFA) verwendet unitäre Übergangsmatrizen $U \in \mathbb{C}^{n \times n}$ statt binärer Adjazenzmatrizen [19]. Die komplexwertige Signatur lautet: $\sigma_j^{(\mathbb{C})} = \sum_i U_{ij} \cdot \omega^i + j \cdot \omega^n$ mit $\omega = e^{2\pi i/n}$ (Einheitswurzel).

Forschungsfragen:

- Gilt Satz 3 für QDFAs? Vermutung: Ja, mit Quanten-LCS-Algorithmen in $O(n^{1.5})$ (Quantenspeedup auf LCS ist bekannt).
- Wie definiert man “Quanten-Subgraph-Inklusion” für Übergangsmatrizen mit komplexen Einträgen?
- Gibt es eine Quanten-Variante des Pumping-Lemmas?

Die Verbindung zu [19] und zur Fehlerkorrektur [18] eröffnet möglicherweise eine Quanten-Grammatiktheorie, die klassische Sprachklassen durch Quanteninterferenz erweitert.

Literatur

- [1] Chomsky, N. (1956). Three models for the description of language. *IRE Transactions on Information Theory*, 2(3), 113–124.
- [2] Chomsky, N. (1959). On certain formal properties of grammars. *Information and Control*, 2(2), 137–167.
- [3] Sipser, M. (2013). *Introduction to the Theory of Computation*, 3rd ed. Cengage Learning.
- [4] Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). *Introduction to Automata Theory, Languages, and Computation*, 3rd ed. Addison-Wesley.
- [5] Turing, A. M. (1936). On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, 42(2), 230–265.
- [6] Kleene, S. C. (1956). Representation of events in nerve nets and finite automata. In C. E. Shannon & J. McCarthy (Hrsg.), *Automata Studies*, 3–41. Princeton University Press.
- [7] Kuroda, S. Y. (1964). Classes of languages and linear-bounded automata. *Information and Control*, 7(2), 207–223.
- [8] Cook, S. A. (1971). The complexity of theorem proving procedures. *Proceedings of the 3rd ACM Symposium on Theory of Computing*, 151–158.
- [9] Gold, E. M. (1967). Language identification in the limit. *Information and Control*, 10(5), 447–474.
- [10] Coppersmith, D., & Winograd, S. (1987). Matrix multiplication via arithmetic progressions. *Proceedings of STOC 1987*, 1–6.
- [11] Courcelle, B. (1990). The monadic second-order logic of graphs I. *Information and Computation*, 85(1), 12–75.
- [12] Bodlaender, H. L. (1998). A partial k -arboretum of graphs with bounded treewidth. *Theoretical Computer Science*, 209(1–2), 1–45.
- [13] Booth, K. S. (1980). Lexicographically least circular substrings. *Information Processing Letters*, 10(4–5), 240–242.
- [14] Epp, S. (2026). Der Subgraph Algorithmus – Lösung des Subgraph-Isomorphismus in $O(n^3)$. Universität Bielefeld. <https://gitlab.com/epp-group/subgraph>
- [15] Epp, S. (2026). Boolesche Matrizenmultiplikation (bool-mm) – Formale Theorie und Anwendungen. Google Drive (epp-group).
- [16] Epp, S. (2026). Polynomielle Reduktionen von Compiler-Problemen auf Subgraph-Isomorphismus. Universität Bielefeld. <https://github.com/hjstephan86/science/tree/main/science/ccl>
- [17] Epp, S. (2026). UML-zu-Code-Generierung via graphbasierter Strukturanalyse. Google Drive (epp-group).

- [18] Epp, S. (2026). Gitterbasierte Kryptographie und Quanten-Fehlerkorrektur. Google Drive (epp-group).
- [19] Epp, S. (2026). Quantenzustände, Verschränkung und Kohärenz. Universität Bielefeld.
- [20] Epp, S. (2026). Lernbasiertes autonomes Netzwerkmanagement in heterogenen Mobilfunknetzen. Google Drive (epp-group).
- [21] Epp, S. (2026). Atmosphärische Systemtheorie – Graphentheoretische Formalisierung der Zirkulationsmuster. Google Drive (epp-group).